

Appendix C: Automated Test Suite Inventory

Appendix C: Automated Test Suite Inventory I

This appendix summarizes the **categories** of tests behind the counts in `??`. Aggregate figures are injected at build time (**1197** tests, **64** files, **9** domain packages under `src/`; 95.96% line-and-branch coverage on `src/` (from `coverage.json`); see `src/generate_manuscript_metrics.py` and `output/metrics.json`). API summary: `docs/api_reference.md`. The open-source package `[@cognitive_case_diagrams2026code]` holds the implementation exercised by these tests. Every test uses real mathematical computations—no mocks or fakes.

- ▶ **Categorical axiom tests:** Identity morphism existence, composition associativity, weight invariants, `is_well_formed()` full axiom check
- ▶ **Enriched category tests:** Hom-value constraints, composition inequality, categorical magnitude, magnitude deficit, full composition check, role clustering

Appendix C: Automated Test Suite Inventory II

- ▶ **Diagram type tests:** Pregroup diagrams validated for `dom == Ty()` and `cod == s`, correct box counts, diagram equality
- ▶ **Metrics tests:** Normal form preservation, depth computation with graceful fallback for pregroup diagrams
- ▶ **Natural transformation tests:** Component morphism construction, `naturality_holds` / `verify_naturality` on identity and incomplete maps, identity transformation generation, vertical composition of transformations, completeness checking over domain objects
- ▶ **Complexity metrics tests:** DisCoPy box/cup/cap counting on transitive/ditransitive diagrams, normal form computation and snake equation verification, syntactic complexity scoring with configurable weights, cross-diagram comparison utilities

Appendix C: Automated Test Suite Inventory III

- ▶ **Topos theory tests:** Geometric theory construction from standard and minimal case categories, classifying topos invariant computation, Morita equivalence verification (positive and negative cases), bridge transfer between equivalent theories with transfer-blocking for non-equivalent theories, enriched theory construction
- ▶ **Fluid-S tests:** Volitional/non-volitional mapping, probability splits, Bats language examples, kernel computation, enriched weight scaling

Appendix C: Automated Test Suite Inventory IV

- ▶ **Active inference tests** (tests/test_cognitive_*.py): Belief construction and entropy, KL divergence (Gibbs' inequality, asymmetry), variational free energy, Bayesian belief update with zero-likelihood edge case, sequential multi-word belief update (five-step generative loop with entropy convergence), prediction error scaling including P600 ERP prediction with boundary weights, expected free energy decomposition (epistemic vs pragmatic), magnitude-based garden-path reanalysis cost with symmetry, N400 semantic violation proxy (including test_cognitive_integration.py)
- ▶ **Quantum case tests**: Crisp POVM orthogonal projectors, graded proto-role POVM, Fluid-S basis rotation, density matrix creation, ?? (in ??) $P(c \mid \rho) = \text{Tr}(E_c \rho)$ verification

Appendix C: Automated Test Suite Inventory V

- ▶ **Cognitive security tests:** Type-violation detection, case frame validation, injection score computation, magnitude-based topological robustness, composition inequality as security boundary
- ▶ **Ditransitive tests:** Three-argument sentence creation, NOM/ACC/DAT case assignment, DisCoPy diagram with three cups, complexity comparison with transitive
- ▶ **Visualization tests** (tests/test_visualization_*.py): Category graphs, enriched heatmaps, functor panels, string and DisCoPy diagrams, complexity and DAIF plots, quantum and security plots, Fluid-S landscapes, syntactic panels—PNG output and structural checks where applicable
- ▶ **DAIF subpackage tests** (224 tests across 8 test files):
 - ▶ test_daif_core.py: Distributional Bellman operator, push-forward return, C51 categorical projection
 - ▶ test_daif_quantile.py: QR-DQN quantile Huber loss, IQN risk distortion (neutral/optimistic/pessimistic/CVaR), Wasserstein distances W_1/W_2

Appendix C: Automated Test Suite Inventory VI

- ▶ `test_daif_inference.py`:
`distributional_case_assignment()` posterior convergence, variational message passing, Bethe free energy, expected information gain
- ▶ `test_daif_prediction.py`: DPE precision-weighting, N400/P600 amplitude from return distributions, full ERPPProfile waveform generation and peak latency
- ▶ `test_daif_policy.py`: `G_policy()` EFE + risk term, Boltzmann policy temperature scaling, distributional epistemic value
- ▶ `test_daif_metrics.py`: Convergence diagnostics (monotonicity, relative reduction), distributional KL divergence, quantile calibration error, return entropy
- ▶ `test_daif_types.py`: `DistributionalReturn` helpers, `DAIFResult` / `ERPPProfile` properties (with integration coverage of re-exported DAIF entrypoints)

Appendix C: Automated Test Suite Inventory VII

► **Cross-module and structural coverage tests**

(`test_cross_module_coverage.py`): Integration paths across enriched category and `case_category` modules not reached by per-module unit tests; verifies composition chaining, morphism weight transitivity (`??`), and magnitude consistency across module boundaries

► **Property-based and parametric tests**

(`test_property_based.py`): Algebraic invariants exercised over parametric inputs—enriched composition inequality, magnitude positivity, and morphism weight bounds in $[0, 1]$ —confirming structural properties hold generically rather than only for specific examples used in unit tests

Appendix C: Automated Test Suite Inventory VIII

► **Visualization multi-module tests**

(`test_visualization_plot_modules.py`,
`test_visualization_syntactic_coverage.py`):

Multi-module rendering pipelines and syntactic panel coverage paths not hit by per-module visualization tests; verifies that all registered plot functions execute to file without exception

► **Script and metrics tests** (`test_diagrams_generator.py`, `test_generate_manuscript_metrics.py`): Round-trip invocation of `scripts/generate_diagrams.py` domain dispatcher and `src/generate_manuscript_metrics.py`; verifies domain registry completeness, figure-path collection, and manuscript variable extraction against the live test suite and installed package versions